SMGW-Leser auf Tasmota mit TinyC

Bau- und Bedienungsanleitung · Stand 09.11.2026 (v007) · Boards ESP32-S3-ETH oder ESP32-P4-WIFI6-POE-ETH · Tasmota 16.0.0.1 mit TinyC (github.com/gemu2015/Sonoff-Tasmota, Zweig universal) · Skripte `slot0-smgw` v016 und `slot1-smgwweb` v009 · Seite `smgw.html` v053 · Ohne persönliche Daten: alle Zugangsdaten, Kennungen und Adressen aus dem eigenen Netz sind Platzhalter oder ausgeblendet
Erstellt mit Hilfe von Claude

> **Nutzung für den eigenen Verbrauch und auf eigene Verantwortung.**
>
> - Die Firmware muss **selbst kompiliert** werden; es gibt keine fertige Firmware zum Herunterladen.
> - Der SMGW-Leser ist **nur mit der in dieser Anleitung genannten Hardware** (Waveshare ESP32-S3-ETH oder ESP32-P4-WIFI6-POE-ETH, Abschnitt 2) und einem **PPC**-Gateway an der HAN-Schnittstelle getestet.
> - Ohne Gewähr: keine Haftung für Schäden, Datenverlust, falsche oder fehlende Werte oder eine Sperre des Gateways nach Fehlversuchen.
> - Die Werte sind eine Kopie dessen, was das Gateway anzeigt. Für Abrechnungen gelten allein die Daten des Messstellenbetreibers bzw. Lieferanten.

> **Eigenbau, nicht zertifiziert.** Der SMGW-Leser liest nur die Werte, die das Smart-Meter-Gateway über seine Kundenschnittstelle (HAN) ausgibt. Er schreibt nichts in das Gateway und ändert nichts am Zähler. Die Zugangsdaten zur HAN-Schnittstelle gehören dem Anschlussnutzer und dürfen nicht weitergegeben werden.

> **Neu gegenüber v006 der Anleitung (09.11.2026 vormittags):**
>
> - **Zweites Board getestet: Waveshare ESP32-S3-ETH** (W5500-LAN, WLAN im Chip, SD-Karte). Es reicht für den SMGW-Leser und ist seit 09.11. unser fester Leser (Abschnitte 2, 4.2, 4.3).
> - `slot1-smgwweb` v009: Bei der **Erstinbetriebnahme** lehnte die Seite die Identifikationsnummer ab („braucht Slot 1 v006“), weil Slot 1 bei noch leerem Zugang kein Feld `id` meldete. Behoben.
> - `slot0-smgw` v016: Lässt das Gateway eine Verbindung still fallen, wiederholt der Leser jetzt auch die **Anmeldung** einmal mit neuer Verbindung (vorher nur den Tagesabruf). Ursache belegt mit der Diagnosezeile aus v015 (Abschnitt 10).
> - Bild 1 ohne Adresse des Lesers.

1. Was der SMGW-Leser macht

Ein Smart-Meter-Gateway (SMGW) speichert die **amtlichen 15-Minuten-Zählerstände** des Zählers: Bezug (OBIS 1-0:1.8.0) und Einspeisung (1-0:2.8.0). Über die HAN-Schnittstelle kann der Anschlussnutzer sie im Browser abrufen. Der SMGW-Leser macht das automatisch:

- holt einmal täglich ab 00:20 die fehlenden ganzen Tage (beim ersten Mal alle Tage ab dem ersten Tag, den das Gateway hat),
- legt sie als Textdateien auf der SD-Karte ab, Werte unverändert wie vom Gateway (4 Nachkommastellen),

- zeigt sie auf einer eigenen Seite als Tag, Monat, Jahr und Jahre mit Diagramm,
- holt auf Wunsch zusätzlich den laufenden Tag in einem einstellbaren Takt, nach der Uhr des Geräts (z. B. Takt 60 → jeweils hh:02),
- liest einmal am Tag die Firmware des Gateways und die freigeschalteten Tarifanwendungsfälle (TAF) mit,
- gibt alle Daten als CSV und über eine fertige Power-Query-Abfrage für Excel aus,
- sichert die Daten jede Nacht auf einen FTP-Server (z. B. USB-Stick an der FRITZ!Box) und holt sie bei Bedarf zurück,
- meldet sich mit Benutzer/Passwort (Digest) oder mit einem Client-Zertifikat an.



Seite `/smgw`, Reiter „Tag“: Tageswerte und Leistung je Viertelstunde (Bezug rot nach oben, Einspeisung gelb nach unten). Adresse des Lesers und Zählerstände für das Bild ausgeblendet.

2. Hardware

Getestet sind zwei Boards von Waveshare. Beide haben LAN (zur HAN-Buchse), WLAN (ins Heimnetz) und einen Steckplatz für die SD-Karte. Der Leser braucht wenig: gemessen rund 0,3 MB PSRAM und gut 30 KB für das Skript; die Tagesseite des Gateways (rund 170 KB) wird in 1-KB-Stücken ausgewertet. **Empfehlung: das ESP32-S3-ETH**, der P4 ist für diese Aufgabe überdimensioniert.

2.1 Waveshare ESP32-S3-ETH (empfohlen)

Teil	Angabe
Board	Waveshare **ESP32-S3-ETH** (SKU 28972); mit PoE-Modul als Satz **ESP32-S3-POE-ETH** (SKU 28771) Kaufen: https://www.waveshare.com/esp32-s3-eth.htm Wiki: https://www.waveshare.com/wiki/ESP32-S3-ETH
Prozessor	ESP32-S3R8 (Revision v0.2), 240 MHz
Speicher	8 MB PSRAM, 16 MB Flash (am Gerät gelesen mit `Status 4`)
WLAN	im ESP32-S3 eingebaut (2,4 GHz), kein Zusatzchip, kein Update nötig
Ethernet	W5500 über SPI (EthType 8): RST GPIO 9, INT GPIO 10, MOSI GPIO 11, MISO GPIO 12, SCLK GPIO 13, CS GPIO 14
SD-Karte	SPI: CS GPIO 4, MISO GPIO 5, MOSI GPIO 6, SCLK GPIO 7 (FAT32)
Strom	USB-C oder PoE (nur mit PoE-Modul). Am HAN-Anschluss: Strom über USB-C, normales Patchkabel direkt zum Gateway.
Gemessen 09.11.2026	Erstabruf 283 Tage in 39 min; danach frei: Arbeitsspeicher 218 KB, größter Block 164 KB, PSRAM 7,7 von 8 MB

2.2 Waveshare ESP32-P4-WIFI6-POE-ETH

Teil	Angabe
Board	Waveshare **ESP32-P4-WIFI6-POE-ETH** (SKU 32832) Kaufen: https://www.waveshare.com/esp32-p4-wifi6-poe-eth.htm?sku=32832 Wiki: https://www.waveshare.com/wiki/ESP32-P4-WIFI6-POE-ETH
Prozessor	ESP32-P4 (Revision v1.3), betrieben mit 360 MHz
Speicher	32 MB PSRAM, 32 MB Flash (am Gerät gelesen mit `Status 4`)
WLAN	ESP32-C6 als Funkchip (ESP-Hosted über SDIO, GPIO 14–19, Reset GPIO 54). **Firmware des C6 mindestens 2.12.6**, siehe Abschnitt 4.4
Ethernet	PHY IP101 (EthType 1), MDC GPIO 31, MDIO GPIO 52, Reset GPIO 51, Takt GPIO 50; Versorgung auch über PoE möglich
SD-Karte	SDIO 4-Bit: D0–D3 = GPIO 39–42, CLK = GPIO 43, CMD = GPIO 44 (FAT32)
Strom	USB-C oder PoE. Wichtig: Kommt der Strom nur über PoE, ist das Board ohne LAN-Kabel aus.
Schaltplan	files.waveshare.com/wiki/ESP32-P4-WIFI6-POE-ETH/ESP32-P4-WIFI6-POE-ETH-Schematic.pdf

> **Kein PoE zwischen Leser und Gateway.** Die HAN-Buchse hat kein PoE. Die Boards nehmen PoE nur auf und geben keine Spannung auf das Kabel. Aber nie einen PoE-Injektor (vor allem keinen passiven) oder PoE-Switch zwischen Leser und Gateway setzen; wird ein Switch gebraucht, dann einen ohne PoE.

3. Netz

Der SMGW-Leser hängt an **zwei Netzen** gleichzeitig:

Anschluss	Netz	Zweck
LAN	direkt (oder über einen eigenen Switch) an der **HAN-Buchse des Gateways**, feste Adresse im Netz der HAN-Schnittstelle, **ohne** Gateway und DNS	nur der Abruf beim SMGW
WLAN	Heimnetz (Adresse per DHCP, am besten im Router fest zuweisen)	Bedienung, Seite, Uhrzeit (NTP), FTP-Sicherung, Excel

Vorgabe (PPC): HAN-Schnittstelle des Gateways `https://192.168.1.200:443/cgi-bin/hanservice.cgi`, SMGW-Leser im LAN fest `192.168.1.21 / 255.255.255.0`. Welche Adresse das eigene Gateway an der HAN-Buchse hat, steht in den Unterlagen des Messstellenbetreibers.

Ändern ohne neue Firmware: Reiter **Einstellungen → Gateway (HAN-Schnittstelle)**. „Adresse des Gateways“ als ganze Adresse mit Port und Pfad eintragen (fehlen sie, gelten 443 und `/cgi-bin/hanservice.cgi`); „Leser im LAN“ ist eine freie Adresse im selben Netz. Ändert sich die LAN-Adresse, startet der Leser einmal neu; über das WLAN bleibt er erreichbar. In der Konsole geht dasselbe mit `SMGWHost`, `SMGWPort`, `SMGWPfad` bzw. `EthIPAddress`.

> Weil im LAN kein Gateway eingetragen ist, bleibt der Weg ins Internet und ins Heimnetz beim WLAN. So können sich die beiden Netze nicht in die Quere kommen. Der Abruf ist für die HAN-Weboberfläche des **PPC**-Gateways gebaut; Gateways anderer Hersteller haben eine andere Oberfläche, dort hilft eine andere Adresse allein nicht.

4. Firmware bauen

4.1 Grundlage

- Tasmota mit **TinyC** von gemu, Zweig `universal`, gebaut mit PlatformIO: **https://github.com/gemu2015/Sonoff-Tasmota/tree/universal**
  TinyC-Beschreibung: `tasmota/tinyc/README_DE.md` und `TinyC_Reference_DE.md` im selben Repository; fertige Firmware und IDE unter **https://github.com/gemu2015/Sonoff-Tasmota/releases** (für den SMGW-Leser trotzdem selbst bauen, Abschnitt 4.2/4.3).
- Der Abruf läuft als TinyC-Skript, nicht als eigener Treiber. Änderungen am Abruf brauchen deshalb kein neues Flashen, nur ein neues Skript.
- Benötigte Stellen in TinyC und tls_mini:
  1. `TC_TLS_RX_BUF` überschreibbar (`#ifndef`) – in `universal` enthalten.
  2. `tc_tls->setRSAOnly(false);` in `SYS_TLS_CONNECT` – in `universal` enthalten. Ohne sie bietet `tlsConnect` nur ECDHE-RSA an, das Gateway (nur ECDSA) bricht den Aufbau sofort ab.
  3. **Nur für die Anmeldung per Zertifikat:** Konsolenbefehl `TinyCTlsCert <Zertifikatsdatei> <Schlüsseldatei>` (`TinyCTlsCert 0` = aus) in `xdrv_124_tinyc.ino` / `xdrv_124_tinyc_vm.h`; `tlsConnect` zeigt das Zertifikat dann vor, wenn der Server danach fragt. Dazu in tls_mini (`WiFiClientSecureLightBearSSL.cpp`): Client-Schlüssel nur setzen, wenn wirklich ein Zertifikat gesetzt ist, und P-256 *und* P-384 erlauben. Wer nur Benutzer/Passwort nutzt, braucht diesen Teil nicht. Wirkt nur in einer Firmware mit `USE_MQTT_CLIENT_CERT` (Abschnitt 4.3); ohne den Schalter antwortet der Befehl ab 09.11.2026 `{"loaded":0,"error":"needs USE_MQTT_CLIENT_CERT"}`.

4.2 PlatformIO-Env (`platformio_override.ini`)

ESP32-S3-ETH: W5500 braucht die SDK-Einstellungen in `custom_sdkconfig`.

```
[env:tasmota32s3-waveshare-ETH_SMGW_TinyC]
extends          = env:tasmota32_base
board_build.f_cpu = 240000000L
board_build.mcu   = esp32s3
board             = esp32s3_16M
build_flags      = ${env:tasmota32_base.build_flags}
                   -DUSE_ETHERNET
                   -DTINYC_TESTING
                   -DTINYC_NO_SCRIPTER
                   -DUSE_BINPLUGINS
lib_extra_dirs   = ${common.lib_extra_dirs}
                   lib/libesp32
                   lib/libesp32_div
lib_ignore       = ${env:tasmota32_base.lib_ignore}
                   TTGO TWatch Library
                   Micro-RTSP
                   epdiy
                   esp32-camera
                   lib/libesp32/CORE2_Library
                   libesp32_lvgl
custom_sdkconfig = CONFIG_ETH_ENABLED=y
                   CONFIG_ETH_USE_SPI_ETHERNET=y
```

```
                    CONFIG_ETH_SPI_ETHERNET_W5500=y
                    CONFIG_LWIP_UDP_RECVMBOX_SIZE=64
                    CONFIG_MBEDTLS_SHA512_C=y
                    CONFIG_FREERTOS_IDLE_TASK_STACKSIZE=3072
custom_component_remove = espressif/esp32-camera
                    espressif/esp_jpg
                    espressif/fb_gfx
                    espressif/esp-dsp
                    espressif/esp_modem
                    espressif/network_provisioning
upload_port      = COMx      ; eigenen Anschluss eintragen
upload_speed     = 460800
```

ESP32-P4-WIFI6-POE-ETH:

```
[env:tasmota32p4-waveshare-WIFI6_SMGW_TinyC]
extends          = env:tasmota32_base
board_build.f_cpu = 360000000L
board_build.mcu   = esp32p4
board             = esp32p4
build_flags      = ${env:tasmota32_base.build_flags}
                   -DUSE_ETHERNET
                   -DTINYC_TESTING
                   -DTINYC_NO_SCRIPTER
                   -DUSE_BINPLUGINS
lib_ignore       = ${env:tasmota32_base.lib_ignore}
                   Micro-RTSP
                   Homekit
                   esp-nimble-cpp
                   esp32-camera
                   JPEG decode
upload_port      = COMx      ; eigenen Anschluss eintragen
```

Kein `custom_sdkconfig` und keine zusätzlichen Bibliotheksordner: Jede dieser Stellschrauben bricht beim P4 die ESP-Hosted-Kette für das WLAN. Das passende Env in `default_envs` eintragen. Das Gerät wird nur über den Block in `user_config_override.h` gewählt, nicht über ein `-D` im Env.

4.3 Geräteblock (`user_config_override.h`)

Ein eigener Block je Board, ausgewählt mit `#define esp32S3_waveshare_ETH_SMGW_TinyC` bzw. `#define esp32p4_waveshare_WIFI6_SMGW_TinyC`. Die wichtigen Zeilen (P4; Unterschiede beim S3 darunter):

```
#define DISABLE_REFERER_CHK          // "HTTP API aktivieren" ab Werk
#define CFG_HOLDER 4641              // eigener Wert -> frische Grundeinstellungen
#define USER_BACKLOG "EthIPAddress 192.168.1.21; EthGateway 0.0.0.0; EthSubnetmask 255.255.2
55.0; EthDNSServer1 0.0.0.0; EthDNSServer2 0.0.0.0; Restart 1"
#define DEVICE_NAME   "SMGW-Leser"
#define FRIENDLY_NAME "SMGW"
#undef  WIFI_DEFAULT_HOSTNAME
#define WIFI_DEFAULT_HOSTNAME "%s"   // Hostname = MQTT-Topic, z. B. "smgw" (immer mit %s!)
#define TC_VM_TASK_STACK 16384       // TLS-Aufbau im Skript braucht Stapel
#define TC_TLS_RX_BUF 16709          // volle TLS-Paketgroesse (16384 + 325)
#define USE_MQTT_CLIENT_CERT         // nur fuer TinyCTlsCert: sonst schickt tls_mini nie ei
n Client-Zertifikat
```

```
#define TC_MAX_HEAP 32768
#define TC_MAX_HEAP_HANDLES 256
#define USE_WEBCLIENT_HTTPS
#define USE_FTP
#define USE_SDCARD
#define USE_UFILESYS
#define USE_ETHERNET
#define ETH_TYPE 1                   // IP101
#define ETH_ADDRESS 1
#define ETH_CLKMODE 0
#define USER_TEMPLATE "{\"NAME\":\"Waveshare ESP32-P4-WIFI6-POE-ETH\", ... }"
```

- **LAN-Adresse:** `USER_BACKLOG` stellt sie beim ersten Start ein (nur bei frischen Einstellungen, also neuem `CFG_HOLDER`) und startet einmal neu. Später auf der Seite änderbar (Abschnitt 3).
- **Hostname:** `"%s"` ergibt den Namen ohne Chip-Nummer; im Router dann z. B. `http://smgw.fritz.box/`. **Nie einen festen Text ohne** `%` eintragen (Absturz beim Start bei gespeicherten Namen mit `%`). Wirkt nur bei frischen Einstellungen; sonst in der Konsole `Hostname smgw`.
- **ESP32-S3-ETH anders:** `ETH_TYPE 8` (W5500), `ETH_ADDRESS 1`, `ETH_CLKMODE 0` und als Vorlage `{"NAME":"Waveshare ESP32-S3-ETH","GPIO":[0,0,0,0,6721,673,705,737,0,5536,5600,704,672,736,5568,0,…],"FLAG":0,"BASE":1}` (LAN- und SD-Pins aus Abschnitt 2.1). Alles andere wie oben. Hängen zwei Leser am selben HAN-Netz (z. B. zum Vergleich), braucht jeder eine eigene LAN-Adresse (bei uns S3 .1.22, P4 .1.21), und es darf immer nur einer abrufen.
- **Vorlage P4:** die vollständige Zeile aus dem WIFI6-Block übernehmen. Die C6-Pins (6, 14–19, 54) sowie 24/25 (USB) und 50 (ETH-Takt) bleiben dort gesperrt (0).
- WLAN-Name und WLAN-Passwort trägt man wie bei jedem Tasmota-Gerät ein (`STA_SSID1` / `STA_PASS1` oder später über die Weboberfläche). Sie gehören **nicht** in veröffentlichte Dateien.

4.4 Funkchip C6 aktualisieren (nur P4)

Neue Boards kommen mit C6-Firmware 0.0.6. Damit hängt jede Abfrage der WLAN-Betriebsart rund 5 s, das ganze Gerät wird träge. Einmalig in der Tasmota-Konsole:

```
HostedOta https://ota.tasmota.com/tasmota32/coprocessor/network_adapter_esp32c6.bin
```

Nach etwa einer Minute meldet das Gerät „Successful. Restarting“. Prüfen mit `Status 2`: `"HostedMCU":{"Hardware":"esp32c6","Version":"2.12.6"}`.

5. Inbetriebnahme

1. Firmware für das passende Board bauen und per USB flashen (ein gebrauchtes Board vorher mit `esptool erase_flash` ganz löschen). Beim ersten Start setzt das Gerät die LAN-Adresse und startet neu. Spätere Versionen gehen per OTA.
2. WLAN einrichten, Adresse im Router nachsehen und fest zuweisen.
3. Nur P4: C6 aktualisieren (Abschnitt 4.4).
4. Einstellungen → Erweitert: **„HTTP API aktivieren“** muss an sein (ist es mit `DISABLE_REFERER_CHK` ab Werk). Ein **„Passwort für Weboberfläche“** setzen (Empfehlung, siehe Abschnitt 6.3).
5. SD-Karte (FAT32) einlegen. Im Log muss beim Start `UFS: SDCard mounted` stehen.
6. Seite `smgw.html` **und** `smgw.html.gz` auf die SD-Karte laden (Abschnitt 8).
7. Skripte laden (TinyC-IDE): `slot0-smgw` in **Slot 0**, `slot1-smgwweb` in **Slot 1**, beide mit Autostart.
8. Seite `http://<Gerät>/smgw` öffnen, Reiter **Einstellungen**: Gateway-Adresse prüfen, **Zugangsdaten** eintragen (Abschnitt 6), Takt wählen.
9. Erster Abruf: „jetzt abrufen“ oder in der Konsole `SMGWAbruf`. Im Log erscheinen `SMG:`-Zeilen. Bei einem Fehler **nicht** sofort wiederholen, sondern erst das Log lesen (Sperre, Abschnitt 7).



Tasmota-Hauptseite: gelber Knopf „SMGW“ (Slot 1), darunter „Gespeichert bis“; ein Fehler erscheint als eigene orange Zeile.

6. Zugang zum Gateway

> **Die Zugangsdaten sind geheim.** Sie liegen nur in Dateien mit der Endung `.secret` auf der SD-Karte des Geräts. Sie gehören nicht in die Firmware, nicht in ein Skript, nicht ins Repo, nicht in Briefe, Foren oder Bildschirmfotos.

6.1 Benutzer und Passwort (Digest)

Reiter **Einstellungen**, goldene Kachel 🔒 **Zugangsdaten der HAN-Schnittstelle**: Benutzer und Passwort aus den Unterlagen des Messstellenbetreibers eintragen, **Zugang speichern**.

- Danach zeigt die Seite nur noch „********* (gespeichert)“, auch für den Benutzernamen. Feld leer lassen = gespeicherter Wert bleibt.
- Der Leser übernimmt neue Daten spätestens nach einer Minute; eine laufende Wartezeit nach einem Fehler (auch die 12 h nach abgelehnter Anmeldung) wird dabei aufgehoben.
- **Identifikationsnummer** (auf dem Gateway, E + 3 Buchstaben + 10 Ziffern): für den Abruf über die PPC-Weboberfläche nicht nötig, vorbereitet für den Maschinenweg (wie TRuDI). Leer = bleibt, „-“ = löschen.
- Gespeichert in `/smgw_zugang.secret`: Zeile 1 Benutzer, Zeile 2 Passwort, Zeile 3 Identifikationsnummer (optional). Wer will, kann die Datei auch am PC anlegen und per Kartenleser kopieren (reiner Text, je höchstens 95 Zeichen, Platzhalter-Beispiel: `BENUTZERNAME-HAN` / `PASSWORT-HAN`).

6.2 Client-Zertifikat (.p12 / .pfx)

Manche Messstellenbetreiber geben statt Benutzer/Passwort ein Zugangszertifikat aus: eine Datei `.p12` oder `.pfx` mit Passwort. In derselben Kachel: Datei auswählen, Passwort der Datei eintragen, **Zertifikat laden**.

- Die Datei wird **im Browser** geöffnet (Bibliothek node-forge, von cdnjs geladen und per Prüfsumme geprüft – der PC braucht dafür kurz Internet). Zum Leser gehen nur Zertifikat und Schlüssel, nicht die Datei und nicht ihr Passwort. **Das Passwort der Zertifikatsdatei wird nirgends gespeichert.**
- Gespeichert als `/smgw_cert.secret` (Zertifikat, DER als Hex) und `/smgw_key.secret` (Kurve, Schlüssel, Ablaufdatum). Die Seite zeigt nur „gespeichert · Kurve · gültig bis …“.
- Nur EC-Schlüssel auf **P-256 oder P-384** (keine Brainpool-Kurven, kein RSA-Schlüssel). Ein falsch eingegebenes Passwort wird abgelehnt („PKCS#12 MAC could not be verified“).
- Bei jeder Sitzung lädt Slot 0 das Zertifikat mit `TinyCTlsCert` und zeigt es vor, wenn das Gateway danach fragt. Antwortet das Gateway gleich mit 200 (Cookie + Token), gilt das als angemeldet; fragt es trotzdem nach Digest, werden Benutzer/Passwort aus 6.1 benutzt.
- **Zertifikat löschen** entfernt beide Dateien; danach `TinyCTlsCert 0`.

> **Getestet** gegen einen eigenen TLS-1.2-Testserver, der ein Client-Zertifikat verlangt: Handshake ECDHE-ECDSA mit P-256, der Server sah das Zertifikat; ohne Zertifikat lehnte er ab.

> **Hinweis: Die Anmeldung per Zertifikat ist an einem echten Gateway nicht getestet.** Uns liegt kein Zugangszertifikat eines Messstellenbetreibers vor – das eigene PPC-Gateway meldet mit Benutzer/Passwort an. Getestet ist nur der Weg im Leser (Zertifikat laden, beim TLS-Aufbau vorzeigen) gegen einen eigenen Testserver. Wer ein Gateway mit Zertifikats-Anmeldung hat: bitte zuerst das Log

> lesen und bei einem Fehler nicht mehrfach wiederholen (Sperre, Abschnitt 7). Rückmeldungen sind willkommen.

6.3 Was die Endung .secret schützt – und was nicht

- Die Firmware sperrt `*.secret` für alle TinyC-Dateiwege übers Netz: nicht herunterladen (Port 82), nicht hochladen/überschreiben (Port 83), nicht über die TinyC-Schnittstelle. Nur die Skripte auf dem Gerät lesen sie.
- Die Seite bekommt Benutzer, Passwort, Identifikationsnummer und Zertifikat nie zurück. Gesendet wird per POST, das Log nennt keine Werte.
- Die FTP-Sicherung nimmt keine `.secret`-Datei mit.
- **Nicht geschützt** ist das Gerät selbst: Ohne „Passwort für Weboberfläche“ kann jeder im Heimnetz die Seite öffnen und neue Zugangsdaten setzen (lesen kann er die alten nicht). Deshalb in Tasmota unter **Einstellungen** → **Erweitert** ein „Passwort für Weboberfläche“ setzen. Wer die SD-Karte in der Hand hat, kann die Dateien lesen.

7. Wie der Abruf abläuft

Schritt	Anfrage an das Gateway
0	TLS 1.2 (ECDHE-ECDSA, P-256, Port 443); ist ein Zertifikat geladen, wird es vorgezeigt, wenn das Gateway danach fragt
1	`GET /cgi-bin/hanservice.cgi` → 401 mit Digest-Nonce (bei Zertifikats-Anmeldung gleich 200)
2	dasselbe mit `Authorization: Digest` (MD5) → 200, Cookie `session`, Token `tkn`
3	`POST tkn=…&action=meterform` → Zählerkennung `mid`
4	je Tag `POST tkn=…&action=showMeterValues&mid=…&from=TAG&to=TAG` (je Antwort ein neues Token)
5	einmal am Tag (erste erfolgreiche Sitzung): `action=swversions` (Software-Versionen) und `action=tariffform` (Auswertungsprofile = freigeschaltete TAF). Nur lesen; scheitert das, zählt der Abruf trotzdem als gelungen.
6	`POST tkn=…&action=logout`

- **Wann:** fehlende ganze Tage ab 00:20; fehlen mehrere Sitzungen lang Tage, mit 10 Minuten Pause zwischen zwei Abrufen. Mit eingeschaltetem Takt zusätzlich der laufende Tag zu festen Uhrzeiten: Abstand nach dem Viertelstundenende + n × Takt ab Mitternacht (Vorgabe Abstand 2 min: Takt 60 → hh:02, Takt 30 → :02 und :32, Takt 15 → :02/:17/:32/:47, 2 h → 00:02, 02:02 …). Ein verpasster Zeitpunkt wird einmal nachgeholt, nie doppelt.
- Alles läuft über **eine** TLS-Verbindung. Jede Anfrage geht genau einmal raus.
- Eine Tagesantwort ist rund 170 KB groß. Sie wird in 1-KB-Stücken gelesen, die Werte bleiben Text (4 Nachkommastellen).
- Das Gateway liefert je Tag die Viertelstunden 00:00 bis 23:45. Der Tagesverbrauch rechnet deshalb bis 00:00 des Folgetages.
- Reißt eine Antwort ab, wird der Tag (auch der laufende) einmal mit neuer Verbindung wiederholt. „Unerlaubter Zugriff“ mitten im Abruf führt zu höchstens zwei neuen Anmeldungen.

> **Sperre:** Das Gateway sperrt nach zu vielen fehlgeschlagenen Anmeldungen (beobachtet: erst 50 s, dann 5 min). Deshalb meldet sich das Skript je Sitzung nur einmal an, wartet nach einem Fehler 60 Minuten (bei jedem weiteren Fehler doppelt so lange, höchstens 24 h) und nach einer abgelehnten Anmeldung 12 Stunden. Fehlversuche nicht von Hand erzwingen.

8. Dateien auf der SD-Karte

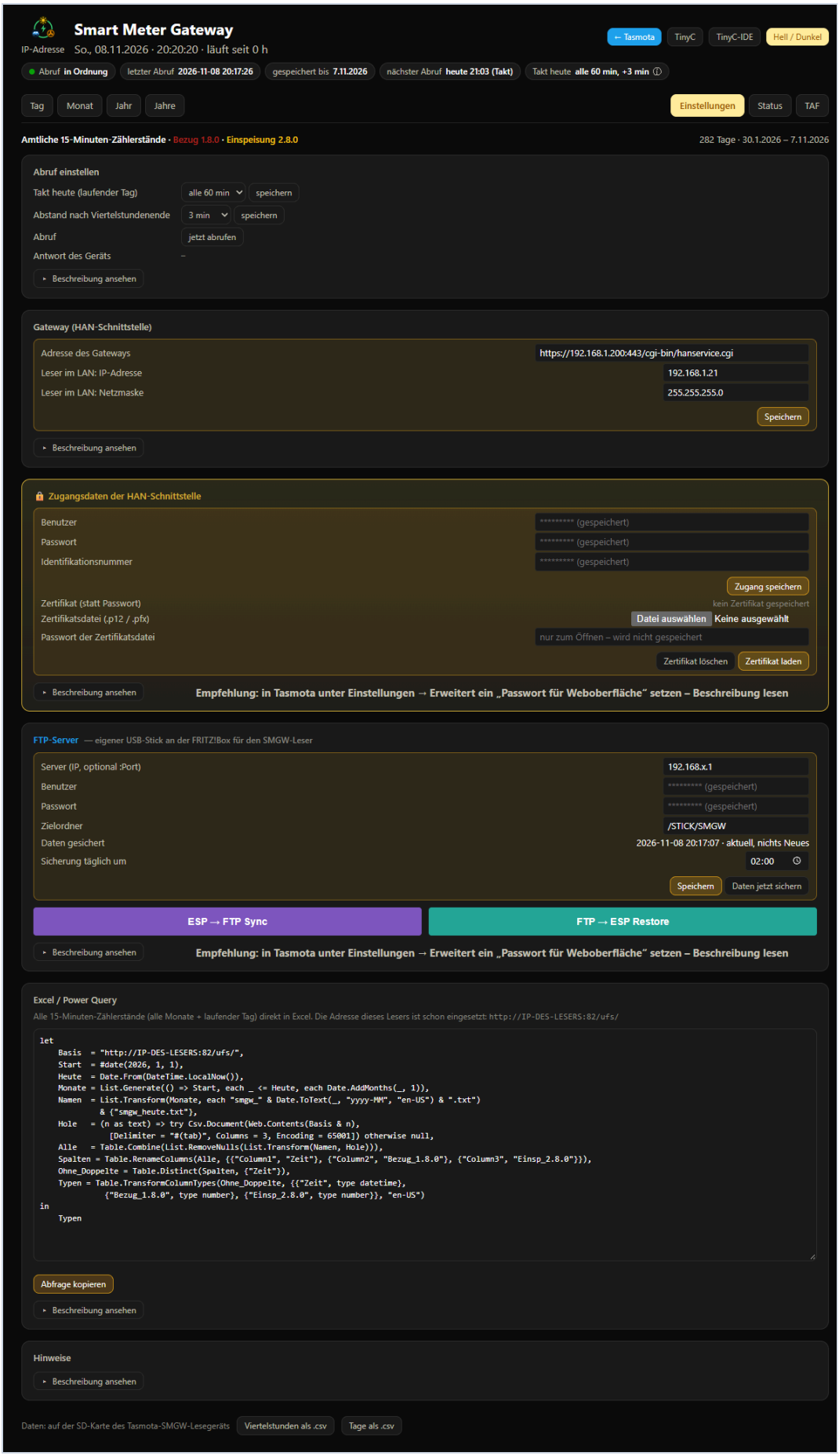
Datei	Inhalt
`/smgw_JJJJ-MM.txt`	je Monat, eine Zeile je Viertelstunde: `JJJJ-MM-TT hh:mm<TAB>Bezug 1.8.0<TAB>Einspeisung 2.8.0` (kWh)
`/smgw_tage.txt`	je Tag: Datum, Bezug Anfang, Bezug Ende, Einspeisung Anfang, Einspeisung Ende, Anzahl Viertelstunden (Anfang = 00:00 des Tages, Ende = 00:00 des Folgetages)
`/smgw_heute.txt`	wie eine Monatsdatei, nur der laufende Tag (nur bei eingeschaltetem Takt)
`/smgw_status.txt`	Zustand als JSON (Zugang, gespeichert bis, letzter Abruf, Fehler, nächster Versuch, Takt, Abstand, Gateway-Adresse, Zertifikat 0/1/2, FTP-Stand, Gateway-Firmware) – nie Zugangsdaten
`/smgw_cfg.txt`	je eine Zeile (alle optional): `takt <min>`, `versatz <min>`, `ftpzeit HH:MM`, `host <IP>`, `port <n>`, `pfad </…>`
`/smgw_gw_fw.html`, `/smgw_gw_taf.html`	die Gateway-Seiten „Software-Versionen“ und „Auswertungsprofil“ im Rohzustand (einmal am Tag, je höchstens 150 KB). Enthalten Kennungen des Gateways – nicht weitergeben.
`/smgw.html` + `/smgw.html.gz`	die Seite; das Gerät liefert zuerst die `.gz` aus. Bei einer neuen Fassung **beide** ersetzen (z. B. `gzip -9 -k -n smgw.html`)
`/smgw_zugang.secret`	Zugang HAN (Abschnitt 6.1)
`/smgw_cert.secret`, `/smgw_key.secret`	Client-Zertifikat und Schlüssel (Abschnitt 6.2), nur wenn benutzt
`/smgw_ftp.secret`	FTP-Zugang, 4 Zeilen: Server · Benutzer · Passwort · Zielordner (Abschnitt 9.2; wird über die Seite gesetzt)
`/smgw_ftp.pos`	Stand der FTP-Sicherung (erster noch offener Monat und Dateigrößen); löschen = beim nächsten Mal alles neu senden

Laden und Sichern der Dateien geht über die TinyC-Dateiports: lesen `http://<Gerät>:82/ufs/sdfs/<Datei>`, schreiben per HTTP-POST an `http://<Gerät>:83/ufs/sdfs/<Datei>`. Die `.secret`-Dateien sind dort gesperrt.

9. Bedienung

- **Tasmota-Hauptseite:** gelber Knopf **„SMGW"**, darunter „Gespeichert bis" und eventuell orange Zeilen „Fehler" und „FTP-Sicherung" (brechen um).
- **Seite `/smgw`:** Kopf mit Adresse, Datum, Uhrzeit und Laufzeit, Knöpfe ← Tasmota, TinyC, TinyC-IDE, Hell/Dunkel. Darunter Abrufzustand, letzter Abruf, gespeichert bis, nächster Abruf, Takt. Reiter:
  - **Tag** (Viertelstunden als Diagramm und Tabelle, Bestenlisten des Jahres), **Monat** (je Tag), **Jahr** (je Monat), **Jahre** (alle Jahre im Vergleich; ein Klick öffnet das Jahr) – jeweils mit Tabelle zum Aufklappen.
  - **Einstellungen:** Abruf (Takt, Abstand, jetzt abrufen), Gateway-Adresse und LAN, Zugangsdaten und Zertifikat, FTP-Server, Excel / Power Query, Hinweise. Jede Kachel hat „Beschreibung ansehen".
  - **Status:** Karten **Abruf**, **Gateway** (Firmware, Werte vorhanden seit) und **Gerät** (Tasmota-Firmware, Prozessor, Funkchip, Skripte, Seite, Speicher, SD-Karte, Netz, Laufzeit, letzter Neustart).
  - **TAF:** die beim Gateway freigeschalteten Tarifanwendungsfälle und Auswerteprofile (live), was dadurch nicht geliefert wird, und eine Erklärung aller 14 TAF.

  Unten auf jeder Seite: **Viertelstunden als .csv** und **Tage als .csv** (UTF-8, Semikolon, Dezimalkomma). Vorzeichen: + Netzbezug, − Einspeisung. Leistung = Zählerdifferenz der Viertelstunde × 4.
- **Daten-Schnittstelle** (für eigene Auswertungen): `/smgwj` Status als JSON, `/smgwf?n=tage|heute|JJJJ-MM|gwfw|gwtaf` die Dateien (nur diese Namen); `/smgwzugang` (POST) Zugänge setzen, Antwort nie mit Benutzer, Passwort oder Schlüssel.



Reiter „Einstellungen" (Zugangsdaten gespeichert, nur „*********" sichtbar). Adressen aus dem eigenen Netz ersetzt.

9.1 Abruf einstellen

Feld	Bedeutung
Takt heute	aus / 15 / 30 / 60 min / 2 h: holt zusätzlich die Viertelstunden des laufenden Tages nach der Uhr. Jeder Takt ist eine eigene Anmeldung am Gateway.
Abstand nach Viertelstundenende	0–10 min (Konsole 0–14), Vorgabe 2: Zeit für das Gateway, die gerade beendete Viertelstunde abzulegen.
jetzt abrufen	holt sofort alle fehlenden Tage (und bei Takt den laufenden). Eine laufende Wartezeit nach einem Fehler bleibt bestehen, der Abruf ist dann vorgemerkt. Unter „Antwort des Geräts" steht während des Abrufs „Abruf läuft …", danach das Ergebnis, z. B. „fertig 10:35:35, in Ordnung – neue Viertelstunden bis 10:30" oder „… keine neuen Werte" (eine Viertelstunde ist erst nach ihrem Ende abrufbar), bei einem Fehler dessen Text.

9.2 Sicherung per FTP

Feld / Knopf	Bedeutung
Server	IP-Adresse des FTP-Servers, optional mit `:Port`. Bei der FRITZ!Box die IP angeben, nicht `fritz.box` (über IPv6 verweigert die Box den passiven Modus).
Benutzer, Passwort	FTP-Benutzer mit Schreibrecht. Leer lassen = gespeicherter Wert bleibt. Die Seite bekommt beide nie zurück.
Zielordner	Ordner aus Sicht dieses Benutzers, z. B. `/STICK/SMGW`; leer = oberste Ebene. Er muss vorhanden sein – das Skript legt ihn nicht an, sondern meldet einen Fehler.
Sicherung täglich um	Vorgabe 02:00 (Tagesabruf 00:20). Jedem Gerät am selben FTP-Server eine eigene Uhrzeit geben (z. B. HEMS 01:00, SMGW-Leser 02:00).
Speichern	schreibt `/smgw_ftp.secret` und die Uhrzeit. Neues Ziel → beim nächsten Lauf geht alles neu hoch.
Daten jetzt sichern	`smgw_tage.txt` und die Monatsdateien, nur geänderte, jeweils ganz; dazu `smgw_cfg.txt`
ESP → FTP Sync	alle Daten noch einmal, die Einstellungen und die Seite (`smgw.html` + `.gz`)
FTP → ESP Restore	holt Daten und Einstellungen zurück, aber nur Dateien, die auf der SD fehlen oder kürzer sind als auf dem Server (z. B. nach einem Kartentausch).

Am besten in der FRITZ!Box einen eigenen Benutzer nur mit NAS-Recht anlegen. Auf dem Server liegen danach normale Textdateien. Fehler (Server nicht erreichbar, Anmeldung abgelehnt, Zielordner fehlt) stehen im Kasten, im Reiter Status und auf der Tasmota-Hauptseite; neuer Versuch nach 1 Stunde.

9.3 Excel / Power Query

Die Kachel **Excel / Power Query** (Reiter Einstellungen) enthält eine fertige Abfrage über alle Monatsdateien und den laufenden Tag. Die Adresse setzt die Seite selbst ein – so, wie sie im Browser aufgerufen wurde (IP oder Name). **Abfrage kopieren**, dann in Excel:

1. Daten → Daten abrufen → Aus anderen Quellen → **Leere Abfrage**
2. Start → **Erweiterter Editor**, alles darin durch die Abfrage ersetzen, Fertig
3. Zugriff: **Anonym**; Datenschutzebenen ignorieren; **Schließen & laden**

4. Aktualisieren: Daten → **Alle aktualisieren** – neue Tage und Monate kommen von selbst dazu.

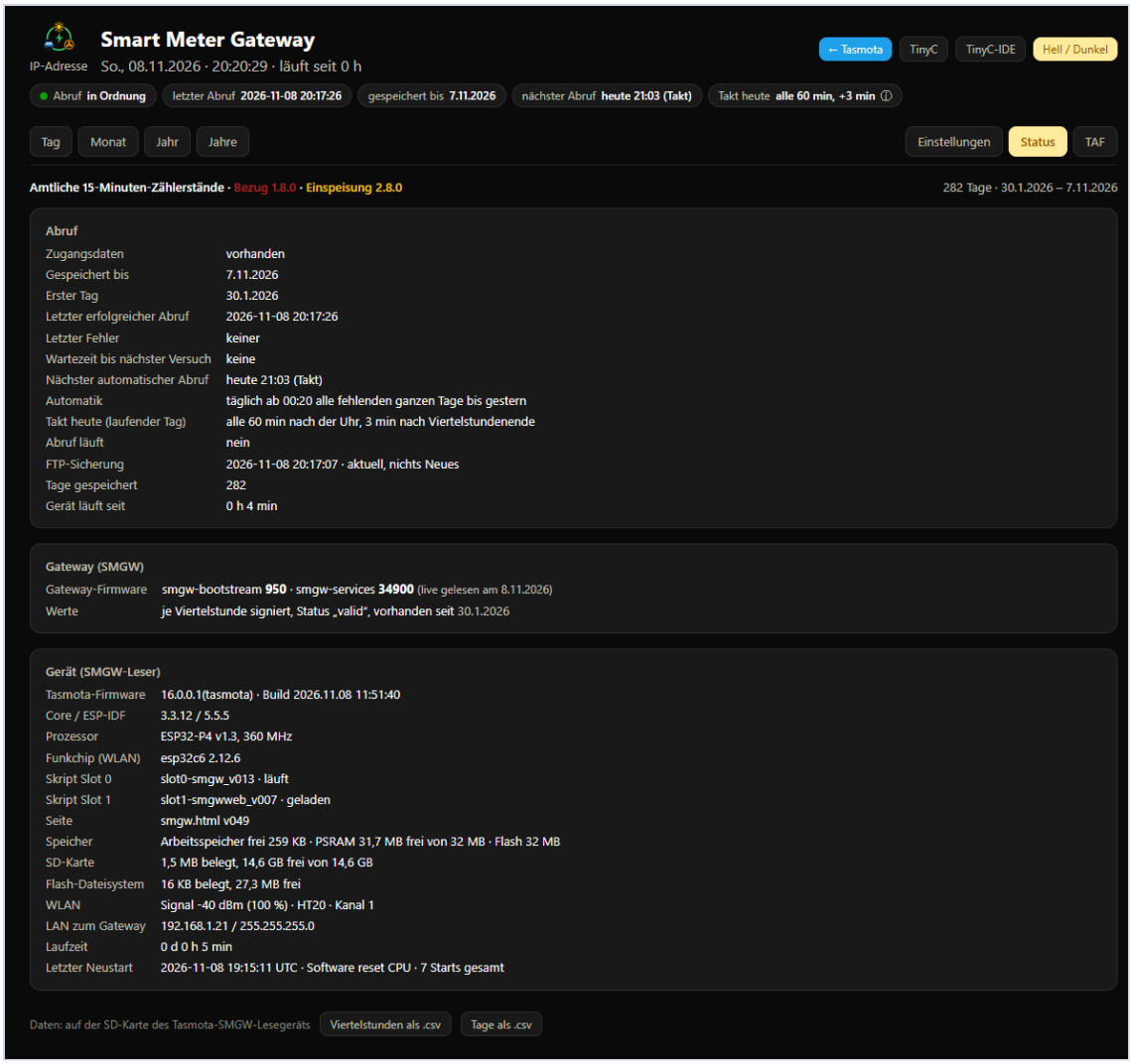
> **Excel holt die Daten vom SMGW-Leser, nicht vom Gateway.** Die Abfrage liest die Textdateien auf der SD-Karte des Lesers (`smgw_JJJJ-MM.txt` , `smgw_heute.txt`) über dessen Port 82, nur lesend, im Heimnetz (WLAN). Mit dem Smart-Meter-Gateway spricht nur der Leser selbst, über sein LAN-Kabel an der HAN-Buchse. Deshalb:
>
> - Excel braucht **keine Zugangsdaten** des Gateways und kein Zertifikat (Zugriff „Anonym“); sie stehen auch nicht in der Abfrage.
> - Ein Aktualisieren in Excel löst **keine Anmeldung am Gateway** aus und kann es nicht sperren – man kann so oft aktualisieren, wie man will.
> - Excel sieht genau den Stand, den der Leser schon abgeholt hat: ganze Tage nach 00:20, den laufenden Tag nur bei eingestelltem Takt (bis zum letzten Takt-Abruf).
> - Der PC muss den Leser erreichen (gleiches Heimnetz), nicht das Gateway.

Die Abfrage, wie sie die Kachel ausgibt (`IP-DES-LESERS` setzt die Seite selbst ein; `Start` = Monat des ersten gespeicherten Tages):

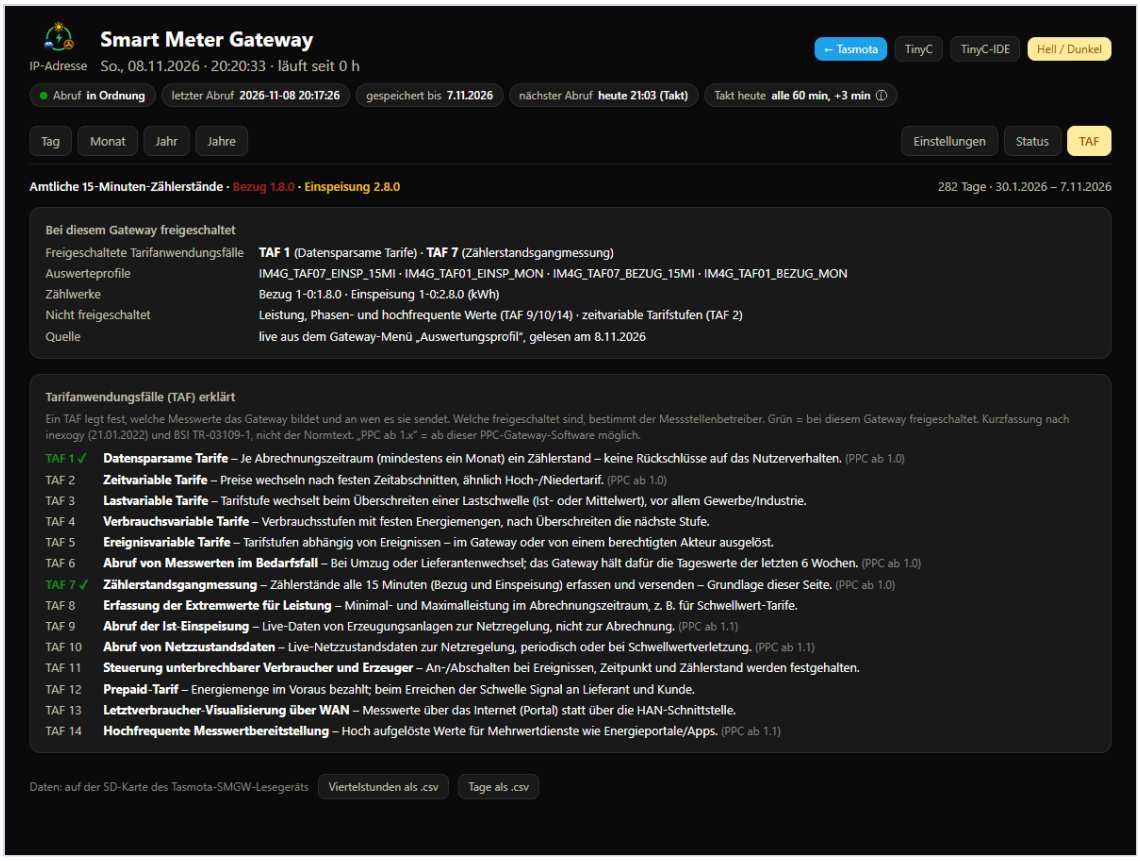
```
let
    Basis  = "http://IP-DES-LESERS:82/ufs/",
    Start  = #date(2026, 1, 1),
    Heute  = Date.From(DateTime.LocalNow()),
    Monate = List.Generate(() => Start, each _ <= Heute, each Date.AddMonths(_, 1)),
    Namen  = List.Transform(Monate, each "smgw_" & Date.ToText(_, "yyyy-MM", "en-US") & ".tx
t")
             & {"smgw_heute.txt"},
    Hole   = (n as text) => try Csv.Document(Web.Contents(Basis & n),
               [Delimiter = "#(tab)", Columns = 3, Encoding = 65001]) otherwise null,
    Alle   = Table.Combine(List.RemoveNulls(List.Transform(Namen, Hole))),
    Spalten = Table.RenameColumns(Alle, {{"Column1", "Zeit"}, {"Column2", "Bezug_1.8.0"},
{"Column3", "Einsp_2.8.0"}}),
    Ohne_Doppelte = Table.Distinct(Spalten, {"Zeit"}),
    Typen = Table.TransformColumnTypes(Ohne_Doppelte, {{"Zeit", type datetime},
              {"Bezug_1.8.0", type number}, {"Einsp_2.8.0", type number}}, "en-US")
in
    Typen
```

Was sie tut: für jeden Monat vom Start bis heute die Monatsdatei holen (fehlende Monate werden übersprungen), dazu die Datei des laufenden Tages; alles zu einer Tabelle zusammenfügen, doppelte Viertelstunden entfernen, Zeit als Datum/Uhrzeit und die Zählerstände als Zahl (Punkt als Dezimalzeichen in der Datei, deshalb „en-US“). Excel zeigt sie dann im deutschen Format.

Spalten: Zeit, Bezug_1.8.0, Einsp_2.8.0 (kWh, Zählerstände; Verbrauch einer Viertelstunde = Differenz zweier Zeilen). Ändert sich die Adresse des Lesers, die Seite unter der neuen Adresse öffnen und neu kopieren – oder nur die Zeile `Basis` ändern.



Reiter „Status“: Abruf, Gateway-Firmware, Gerät. IP-Adresse ausgeblendet.



Reiter „TAF": hier freigeschaltet TAF 1 und TAF 7 (15-Minuten-Zählerstände), darunter alle 14 TAF erklärt.

9.4 Konsole

Befehl	Bedeutung
`SMGW`	Zustand (Zugang, gespeichert bis, letzter Abruf, Fehler)
`SMGWAbruf`	Abruf jetzt vormerken (eine laufende Wartezeit nach einem Fehler bleibt bestehen)
`SMGWTakt <min>`	laufenden Tag alle <min> Minuten nach der Uhr holen (mindestens 15, 0 = aus)
`SMGWVersatz <min>`	Abstand nach dem Viertelstundenende (0–14, Vorgabe 2)
`SMGWHost <IP>` · `SMGWPort <n>` · `SMGWPfad </…>`	Adresse der HAN-Schnittstelle (Vorgabe 192.168.1.200 · 443 · /cgi-bin/hanservice.cgi)
`SMGWFtp` · `SMGWFtp SYNC` · `SMGWFtp RESTORE`	FTP: Daten jetzt sichern · alles senden · zurückholen. Schreibweise egal (`sync` = `SYNC`).
`SMGWFtpZeit HH:MM`	Uhrzeit der täglichen Sicherung
`SMGWFrei`	Wartezeit nach einem Fehler aufheben (nur bewusst benutzen, Sperre!)
`TinyCTlsCert`	Zustand des Client-Zertifikats (`loaded` , Kurve); setzt Slot 0 selbst

Zugangsdaten gibt es absichtlich nicht als Konsolenbefehl.

10. Häufige Fragen

„Fehler: keine Verbindung zum Gateway“

LAN-Kabel, LAN-Adresse (`Status 5`) und die Gateway-Adresse (Einstellungen) prüfen. Der Leser muss im selben Netz wie die HAN-Buchse sein. Der nächste Versuch kommt automatisch nach der Wartezeit.

Woran es genau lag, zeigt im Log die Zeile `SMG: Diagnose <Schritt>: Stufe …` (ab Slot 0 v015): Stufe 1 = Verbindungsaufbau gescheitert (`tlsConnect`), 2 = Senden gescheitert, 3 = keine Antwortzeile, 4 = Antwort unvollständig. Dazu `neu` (Verbindung neu aufgebaut oder weiterverwendet), `stand` (Verbindung bestand noch), `vorher close` (die vorige Antwort hatte „Connection: close“), Status, angekündigte Länge und gelesene Bytes. Die Zeile enthält keine Zugangsdaten.

Typischer Fall: `Stufe 3, neu 0, stand 1, vorher close 0` – das Gateway hat eine weiterverwendete Verbindung still geschlossen. Der Leser holt den Tag dann mit neuer Verbindung („Tag … erneut mit neuer Verbindung“), ab v016 auch die Anmeldung („Anmeldung erneut mit neuer Verbindung“). Beim Erstabruf am 09.11. kam das bei 283 Tagen viermal vor, alle Tage wurden geholt.

„Identifikationsnummer braucht Slot 1 v006“ bei der Ersteinrichtung

Fehler in `slot1-smgwweb` bis v008 bei noch leerem Zugang. `slot1-smgwweb` v009 laden. Umweg mit v008: erst Benutzer und Passwort speichern, Seite neu laden, dann die Identifikationsnummer.

„Anmeldung abgelehnt – Zugangsdaten prüfen“

Benutzername oder Passwort stimmen nicht. Auf der Seite neu eintragen; neue Daten heben die 12-h-Wartezeit auf. Nicht mehrfach hintereinander probieren, sonst sperrt das Gateway.

Die TLS-Verbindung bricht sofort ab

Die Firmware enthält `setRSAOnly(false)` nicht (Abschnitt 4.1). Das Gateway bietet nur ECDSA an.

Beim Zertifikat steht „⚠ Firmware lädt es nicht“

Die Firmware ist älter als 08.11.2026 oder ohne `USE_MQTT_CLIENT_CERT` gebaut (Abschnitt 4.3). Der Abruf läuft dann mit Benutzer/Passwort weiter. Ab Firmware 09.11.2026 antwortet `TinyCTlsCert` in diesem Fall selbst mit `"error":"needs USE_MQTT_CLIENT_CERT"` .

Die Diagramme fehlen, die Tabellen sind da

Die Seite lädt die Diagramm-Bibliothek Chart.js von cdnjs und prüft sie per Prüfsumme; der Browser braucht dafür Internet. Ohne Internet oder bei falscher Prüfsumme fehlen nur die Diagramme, alle Werte stehen weiter in den Tabellen.

Das ganze Gerät ist träge, Seiten laden Sekunden

Nur P4: Der C6 hat noch die Werksfirmware. `HostedOta` ausführen (Abschnitt 4.4).

FTP: „Zielordner fehlt oder kein Schreibrecht“

Der Ordner existiert für diesen Benutzer nicht. Bei einem Benutzer, dessen Freigabe nur auf einen Ordner zeigt, ist dieser Ordner schon „/“ – Zielordner dann leer lassen.

Excel findet nichts

Die Adresse in der Zeile `Basis` stimmt nicht mehr (neue IP des **Lesers** – die Adresse des Gateways spielt für Excel keine Rolle). Seite unter der aktuellen Adresse öffnen und die Abfrage neu kopieren.

An einzelnen Tagen fehlen Viertelstunden

Das Gateway hat sie selbst nicht. Am Tag der Zeitumstellung im Herbst werden die doppelten Viertelstunden überschrieben; die Tagessumme stimmt, nur das Diagramm zeigt 96 statt 100 Werte.